

Design Patterns in academy

Cinci exercitii de aplicare pe codul tau: Factory Method, Strategy, Decorator, Observer.

Cum se lucrează

Ce ai închis deja

fratele lăsat în urmă

de la switch la fabrici

o regulă scrisă de două ori

un jurnal peste mapper

fără hint

Regula de lucru

Cum se lucrează

Cinci exerciții, **în ordinea asta**. Nu e o preferință de-a mea — e impusă de cod, și exercițiul 4 explică de ce. Un exercițiu per commit, cu build verde și aplicația pornită înainte de fiecare.

Toate cele patru pattern-uri le-ai scris deja în `design-patterns-csharp`. Aici nu înveți pattern-uri noi, le muți pe cod adevărat — al tău.

Înainte de fiecare exercițiu: deschide exercițiul corespunzător din `design-patterns-csharp`. E scris acolo, de tine, și merge. Nu-l reconstrui din memorie.

Ce ai închis deja

În `deab1cf` ai reparat singur blocantul care ținea aplicația închisă de șase runde. Am verificat prin rulare, toate trei:

- te loghezi ca student și intri în meniu — **prima dată când aplicația e utilizabilă**;
- o linie `TEACHER,` adăugată cu mâna e recunoscută ca profesor și i se cere parola, deci `TeacherFromText` nu mai e cod mort;
- o linie cu un tip inventat dă `Unknown user type: PARINTE`, nu o eroare generică.

Și `ToText` nu mai scrie `USER,` pe tot — acum aruncă. Aia era coruperea tăcută a lui `users.txt`.

Exercițiul 1 pleacă de aici, de la cod care **merge**.

0. Restanță — fratele lăsat în urmă

`EnrolmentTextMapper` e acum corect: scrii cu `ToString("yyyy-MM-dd")` și citești cu `ParseExact` pe `InvariantCulture`. E mai bine decât ți-am cerut.

Deschide `Books/Mappers/BookTextMapper.cs` și compară-l cu ce tocmai ai scris.

Gata când: cele două mappere citesc data în același fel.

Nu e un pattern, sunt două minute. E aici pentru că e a treia oară când repari o clasă și îi lași fratele neatins — `ex3`, `ex7`, și acum. Când repari ceva, caută cine mai face la fel.

1. Factory Method — de la `switch` la fabrici

`UserTextMapper.FromText` merge acum, cu un `switch` pe `cuv[0]`. Exercițiul ăsta nu repară nimic: ia o soluție care funcționează și o transformă în pattern.

Rescrie-l cu Factory Method, apoi răspunde-mi în scris la întrebarea de la „gata când”.

Hint 1: `ex10` are forma completă — o interfață cu o proprietate `Tip` și o metodă care construiește, plus câte o fabrică per tip. Acolo le-ai ținut într-un array și ai căutat după tip.

Hint 2: întrebarea care separă cele două unelte: **unde se termină switch-ul?** Dacă se termină într-un calcul sau o valoare, ai nevoie de Strategy. Dacă se termină într-un `new`, ai nevoie de Factory Method. Uită-te unde se termină al tău.

Gata când: adaugi un tip nou de utilizator — `PARINTE`, cu un câmp în plus — și îmi arăți **câte fișiere existente ai fost nevoit să modifici**. Apoi spune-mi câte erau cu switch-ul.

Dacă răspunsul e același număr, spune-mi asta. E un rezultat corect al exercițiului.

2. Strategy — o regulă scrisă de două ori

Deschide `Users/Models/Teacher.cs` și `Users/Models/Admin.cs` una lângă alta și compară setterul `Salary` din fiecare. Apoi compară setterul `Password` din fiecare.

Ai două situații diferite în față. Tratează-le diferit.

Hint 1: una dintre cele două perechi e *aceeași regulă cu un număr diferit*. Cealaltă e *exact același cod*. Doar una dintre ele cere un pattern.

Hint 2: înainte să scrii o linie, răspunde-mi în scris la o întrebare: regula de salariu variază **pe rol**, sau doar numărul e altul? Dacă răspunsul tău e „doar numărul”, atunci soluția corectă nu e Strategy — și asta e un rezultat bun al exercițiului, nu un eșec.

Gata când: `grep -rn "minim pe economie" Users` întoarce **un singur** rezultat, iar un profesor cu salariul 100 tot nu poate fi creat.

3. Decorator — un jurnal peste mapper

Scrie un decorator peste `Common/ITextMapper.cs` care, înainte să dea mai departe, scrie în consolă ce linie citește sau ce linie scrie.

Îl pui pe `UserRepository` fără să schimbi nimic în `Repository<T>` și fără să schimbi vreun mapper existent.

Hint 1: decoratorul implementează `ITextMapper<T>` și primește în constructor un `ITextMapper<T>`. Exact forma din `ex11`.

Hint 2: ambele metode au valoare de retur. Un decorator care nu cheamă interiorul n-ar avea ce să întoarcă — **ia asta ca pe regula de verificare**, nu ca pe o coincidență. Când metoda decorată e `void`, regula e aceeași, doar că nimic nu te mai obligă s-o respecți.

Gata când: pornești aplicația, vezi în consolă liniile citite din `users.txt`, și scoți decoratorul cu o singură modificare, într-un singur loc.

4. Observer — salvarea se întâmplă singură

Caută în tot proiectul cine cheamă `Save()`. Vei găsi doar locul unde e **declarat**, în `Common/Repository.cs`.

Asta înseamnă că nimic din ce faci în aplicație nu ajunge vreodată pe disc. Adaugi o carte, închizi, redeschizi: cartea nu mai e.

Fă ca repository-ul să anunțe când s-a schimbat ceva, și scrie doi ascultători: unul care salvează, altul care scrie în consolă ce s-a schimbat.

Hint 1: sursa e `Repository<T>` — `Add` și `Remove` sunt momentele în care se schimbă ceva. Ascultătorii sunt `ex2`, `ex4`, `ex7`, `ex8`, oricare din ele.

Hint 2: doi ascultători, nu unul. Cu unul singur n-ai cum să vezi diferența dintre Observer și un simplu apel la `Save()` pus în `Add`.

De ce exercițiul ăsta e ultimul: e primul moment în care aplicația chiar scrie peste `Data/*.txt`. Până acum, orice greșeală de mapper rămânea în memorie și dispărea la închidere. De aici încolo se scrie pe disc, iar un mapper stricat strică fișierul, nu doar rularea. De-asta vine după ce toate mapper-urile sunt corecte, nu înainte.

Gata când: adaugi o carte, închizi aplicația, o redeschizi, și cartea e acolo. Și `git diff Data/` e gol după o pornire în care n-ai schimbat nimic.

5. Provocare — fără hint

`ViewStudent.cs`, primele linii ale clasei: fiecare `View` își construiește singur serviciile, iar fiecare serviciu își construiește singur repository-ul, care recitește fișierul de pe disc.

Numără câte copii ale lui `books.txt` ai în memorie după ce te loghezi și intri la cărți.

Spune-mi ce se întâmplă cu exercițiul 4 în situația asta, și ce ai schimba.

Nu-ți cer cod la exercițiul ăsta. Îți cer răspunsul, în scris.

Regula de lucru

1. Un exercițiu, un commit. Mesaj scurt, care spune ce pattern ai pus și unde.
2. Build verde **și aplicația pornită** înainte de fiecare commit. Build verde singur nu dovedește nimic — ai avut de două ori build verde cu aplicația inutilizabilă.
3. Rulezi **tot** drumul, nu doar bucata pe care ai atins-o: login → cărți → cursuri → statistici.
4. Testul de 30 de secunde, după fiecare exercițiu: pornești, salvezi, pornești din nou, `git diff Data/` trebuie să fie gol. Prinde regresiile tăcute, care nu crapă.



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro